

Advanced Programming

Exam-Style Revision Questions (Ch.1–Ch.10)

Questions first, then solutions per chapter (Version A)

How to use this sheet

- Do the **Questions** of a chapter first (without looking).
- Then check the **Solutions** and fix your own wording.
- Keep answers **short but precise**: define terms, then apply them.
- When code is given: explain **what happens** (compile-time vs runtime, copy vs move, ownership, etc.).

Contents

1	Chapter 1: Basic Concepts	2
2	Chapter 2: Functions & Namespaces	4
3	Chapter 3: Memory	5
4	Chapter 4: Classes & Memory	7
5	Chapter 5: Advanced Memory	8
6	Chapter 6: Class Design	10
7	Chapter 7: Class Derivation	11
8	Chapter 8: Design Patterns	12
9	Chapter 9: Multi-threading	14
10	Chapter 10: Generic Programming	15

1 Chapter 1: Basic Concepts

Questions

C1-Q1: Compiler phases and timings

Explain the main phases from source file to running program. For each phase, name *what* it produces (e.g., preprocessed source, object file, executable) and *when* it happens (preprocessing / compile time / link time / loading time / runtime).

C1-Q2: Declaration vs definition

Give the difference between a **declaration** and a **definition**. Provide one example that is *only* a declaration and one example that is both a declaration and a definition.

C1-Q3: Binding and binding time

Define **binding**. Give two examples of properties that are bound *before* runtime, and two that are bound *during* runtime.

C1-Q4: Static vs dynamic properties

From a semantics point of view, give two differences between a **static** and a **dynamic** property. Give one example of each.

C1-Q5: Static type vs dynamic type

Explain the difference between **static type** and **dynamic type**. In the code below, state the static and dynamic type of `b`:

```
struct Base { virtual ~Base() = default; };
struct Derived : Base {};

void f(Base& b) {
    // ...
}

int main(){
    Derived d;
    f(d);
}
```

C1-Q6: Linkage

Define **linkage**. Describe the common kinds of linkage you can have for a name in C++ (high level is fine), and why it matters for linking.

C1-Q7: Preprocessor macros vs language features

Give two risks of heavy macro usage (`\#define`) in a large project, and two safer alternatives in modern C++.

Solutions

C1-A1

Typical pipeline:

- **Preprocessor:** expands `\#include`, macros, conditional compilation → *preprocessed source*.
- **Compiler:** parses + type-checks + optimizes → *object file(s)* (machine code + relocation info).
- **Linker:** resolves external symbols across object files and libraries → *executable* (or shared library).
- **Loader:** OS maps executable + shared libs into memory, sets up process → ready to run.
- **Runtime:** program executes; dynamic allocations, virtual dispatch, I/O, etc.

C1-A2

A **declaration** introduces a name and its type so the compiler can accept uses of it. A **definition** provides the actual entity (storage or body) so the linker/runtime has something to use.

- Declaration-only example: `extern int g;`
- Declaration & definition example: `int g = 0;` (allocates storage) or `int f(){return 1;}` (provides body).

C1-A3

Binding = associating a property with a name.

- Before runtime (static): expression types, overload selection, template instantiation, addresses of global functions (often at link time).
- During runtime (dynamic): values stored in variables, dynamic dispatch target (virtual call), memory returned by `new`.

C1-A4

Static properties are determined from the source without executing (e.g., static type, well-formedness, which overload is selected). Dynamic properties happen during execution and may depend on runtime data (e.g., which branch executes, the dynamic type behind a base reference, actual heap addresses).

C1-A5

Static type is the compile-time type of an expression; dynamic type is the most-derived runtime type of the referred object. In `f(Base& b)`, `b` has static type `Base` but dynamic type `Derived` (because it refers to `d`).

C1-A6

Linkage describes whether different occurrences of a name refer to the *same* entity across translation units. Common kinds: **external linkage** (usable across TUs, typical for non-`static` globals), **internal linkage** (restricted to one TU, e.g., `static` at namespace scope), and **no linkage** (local variables). It matters because the linker must decide which references match which definitions.

C1–A7

Risks: (1) no type checking, surprising precedence/side effects; (2) name collisions and hard-to-debug errors across translation units. Safer alternatives: **constexpr** variables/functions, inline functions, templates, scoped enums, and **std::min/std::max** instead of macro versions.

2 Chapter 2: Functions & Namespaces**Questions****C2–Q1: Inline vs constexpr**

Explain the difference between **inline** and **constexpr** functions. Include one advantage and one disadvantage of each.

C2–Q2: [[nodiscard]]

What does `[[nodiscard]]` mean? Give two realistic situations where it is useful.

C2–Q3: noexcept and performance

Explain what **noexcept** promises. Why can **noexcept** improve performance in standard containers?

C2–Q4: Overload resolution conversion ranking

Describe the typical conversion sequence ranking used when selecting the best overload. List the main categories from best to worst.

C2–Q5: Function prototype vs signature

Define: **prototype** and **signature**. What is included in a function signature in C++?

C2–Q6: Namespaces and name conflicts

Why do namespaces exist? Show a small example where two libraries define the same name and how namespaces fix the conflict.

C2–Q7: using-directive vs using-declaration

Explain the difference between `using namespace X;` and `using X::name;`. Which one is safer in headers, and why?

Solutions**C2–A1**

inline suggests inlining (no call overhead) and also allows the same definition in multiple TUs (important for header-defined functions). Downside: can increase binary size. **constexpr** means the function *can* be evaluated at compile time when given constant arguments; otherwise it runs at runtime like a normal function. Benefit: can remove runtime computation; downside: restrictions and may increase compile time.

C2–A2

`[[nodiscard]]` asks the compiler to warn if a return value is ignored. Useful for: (1) error codes or status objects that must be checked; (2) pure computations where ignoring the result is likely a bug (e.g., `std::string s = make();` vs calling `make();` and discarding).

C2–A3

noexcept states: this function will not throw exceptions. With that guarantee, containers like `std::vector` prefer moving elements during reallocation if move is **noexcept**; otherwise they may copy to preserve strong exception safety.

C2–A4

Typical ranking (best → worst):

1. Exact match (no conversion / trivial)
2. Promotion (e.g., `short→int`, `float→double`)
3. Standard conversion (e.g., `int→double`)
4. User-defined conversion

C2–A5

Signature: function name + parameter types (and sometimes qualifiers for member functions).
 Prototype: return type + signature (i.e., full declaration needed by compiler).

C2–A6

Namespaces group names to avoid collisions. Example: `libA::print()` and `libB::print()` can both exist; callers choose explicitly: `libA::print();`.

C2–A7

`using namespace X;` imports *everything* from `X` into the current scope (more collisions). `using X::name;` imports only one name. In headers, avoid `using namespace ...` because it leaks into all inclusions.

3 Chapter 3: Memory

Questions

C3–Q1: Stack vs heap

Explain the difference between stack and heap allocation. Give one example where stack allocation is clearly better, and one where heap allocation is required.

C3–Q2: Storage duration vs lifetime

Define **storage duration** and **lifetime**. Are they always the same? Explain with one clear example.

C3–Q3: Four storage durations

List the four storage durations in C++ and briefly describe when memory is acquired and released for each.

C3–Q4: Static memory area

What is stored in the static memory area? Mention both read-only and writable parts.

C3–Q5: Dangling pointer

Give a short code fragment that produces a dangling pointer on the stack. Explain what goes wrong.

C3–Q6: new/delete correctness

What is a memory leak? What is a double delete? Give one best-practice rule to avoid both.

C3–Q7: Memory diagram reasoning

Consider:

```
int* p = new int{42};
int* q = p;
delete p;
// use q here
```

What is the state of `p` and `q` after `delete p`? Why is using `q` undefined behavior?

Solutions**C3–A1**

Stack: automatic storage, very fast, freed at end of scope. Heap: dynamic storage, flexible lifetime/size, slower allocation. Stack is better for small local objects with clear scope. Heap is needed when lifetime must outlive the current scope or size is decided at runtime (e.g., large data shared across components).

C3–A2

Storage duration = rules for allocation/free of the storage. Lifetime = time the object can be referenced consistently. Often they align, but they can differ (e.g., storage exists but object lifetime ended, or lifetime is extended via reference binding in some cases). Simple example: `int* p = new int{1}`; pointer has automatic storage duration, but the pointed `int` has dynamic storage duration; their lifetimes differ.

C3–A3

- Automatic: start of scope → end of scope.
- Static: program start → program end.
- Thread: thread start → thread end.
- Dynamic: `new`/allocation → `delete`/deallocation.

C3–A4

Static memory area includes read-only code/const data (machine code, string literals, constants) and static data (globals/static variables) that exist for the whole program.

C3–A5

```
int* bad(){
    int x = 5;
    return &x; // returns address of local
}
```

`x` dies at end of scope; returned pointer points to invalid memory.

C3–A6

Leak: allocated memory is never freed. Double delete: deleting same allocation twice. Best practice: avoid raw owning pointers; use RAII (`std::unique_ptr`, `std::vector`, `std::string`).

C3–A7

After `delete p`, the heap int is freed. `p` and `q` still hold the old address, but it no longer refers to a valid object. Using `q` is UB because it is a dangling pointer.

4 Chapter 4: Classes & Memory

Questions

C4–Q1: Constructor responsibilities

Give three goals/tasks of a constructor in well-designed code. Why is establishing the class invariant crucial?

C4–Q2: Copy constructor vs copy assignment

Explain the difference between copy constructor and copy assignment operator, in terms of *memory state* (already constructed vs not).

C4–Q3: Member initializer list

Why is a member initializer list preferred over assigning inside the constructor body? Give two reasons.

C4–Q4: Rule of 0/3/5

State the Rule of 0/3/5. When do you need to care about it?

C4–Q5: default and delete

What does `= default` do? What does `= delete` do? Give one example use case for each.

C4–Q6: Destructor and ownership

When should a destructor manually `delete` a pointer member? What is the modern alternative?

C4–Q7: Predict operations

Assume `Foo` prints messages in default ctor, copy ctor, copy assignment, and dtor. For each fragment, state which operations definitely happen:

```
Foo a;
Foo b = a;

Foo c;
c = a;
```

Solutions

C4–A1

Constructor goals: (1) initialize members, (2) acquire resources (if any), (3) establish class invariant (object starts valid). Invariant matters because every later method assumes it; if ctor fails to establish it, any method can break or become unsafe.

C4–A2

Copy constructor creates a *new* object from another: it constructs and initializes memory. Copy assignment overwrites the state of an *already constructed* object.

C4–A3

Initializer list constructs members directly (no default-construct then assign). It is required for references/const members and often more efficient and correct.

C4–A4

Rule of 0: prefer types that manage resources via RAII so you write no special members. Rule of 3: if you define destructor/copy ctor/copy assignment, likely need all three. Rule of 5 adds move ctor/move assignment for modern C++.

C4–A5

`= default` asks compiler to generate the usual member-wise operation. `= delete` forbids usage. Example: default a default ctor when you also declared other ctors; delete copy operations to make a type non-copyable.

C4–A6

Only delete if the class *owns* the pointed object. Modern alternative: store ownership explicitly with `std::unique_ptr` or `std::shared_ptr`.

C4–A7

`Foo b = a;` calls copy constructor (copy-initialize). `c = a;` calls copy assignment (object already exists).

5 Chapter 5: Advanced Memory

Questions

C5–Q1: Ownership definition

Define **ownership**. Who is responsible for cleanup? Why is unclear ownership dangerous?

C5–Q2: `unique_ptr` vs `shared_ptr`

Compare `std::unique_ptr` and `std::shared_ptr` in terms of ownership, copying, and when the object is deleted.

C5–Q3: Does moving a `unique_ptr` move the object?

Consider:

```
auto p1 = std::make_unique<int>(42);
auto p2 = std::move(p1);
```

What changes in memory/ownership? Does the `int` move?

C5–Q4: Value categories

Name the main value categories (lvalue, xvalue, prvalue) and explain what they mean practically.

C5–Q5: Copy vs move (argument view)

From the point of view of arguments received: what is the difference between copy and move operations? Mention typical parameter types.

C5–Q6: RAII

Define **RAII**. Give one example using a standard library type that follows RAII.

C5–Q7: shared_ptr cycles

Explain what a **reference cycle** is with **shared_ptr**. How do we solve it in C++?

Solutions**C5–A1**

Ownership means a piece of code controls the lifetime of an object and must ensure cleanup. Unclear ownership leads to leaks (nobody deletes) or double delete (multiple delete).

C5–A2

unique_ptr: exclusive ownership, non-copyable, movable; deletes object when the **unique_ptr** is destroyed. **shared_ptr**: shared ownership via reference counting, copyable; deletes object when the last **shared_ptr** owning it is destroyed.

C5–A3

After move: **p2** owns the same heap **int**; **p1** becomes empty (null). The **int** object stays at the same address; only the owning pointer wrapper is moved.

C5–A4

lvalue: has identity/address, can appear on left of assignment. **rvalue**: temporary value (pure rvalue), often materialized as temporary. **xvalue**: expiring value, can be moved-from (e.g., **std::move(x)**).

C5–A5

Copy typically takes **const T&** (source must remain valid). Move typically takes **T&&** (rvalue reference), allowing stealing resources and leaving source in valid-but-unspecified state.

C5–A6

RAII: acquire resources in constructors, release in destructors. Example: **std::lock_guard<std::mutex>** locks in ctor and unlocks in dtor; also **std::vector** manages heap memory automatically.

C5–A7

A cycle: A owns B and B owns A via **shared_ptr**, so refcount never reaches 0 → leak. Fix: break ownership with **std::weak_ptr** for back-references.

6 Chapter 6: Class Design

Questions

C6–Q1: Interface and ADT

What is a class interface? What does it mean for a class to form an **Abstract Data Type (ADT)**?

C6–Q2: Invariants

Define an **invariant**. Give a small example invariant for a class (in words is fine).

C6–Q3: Physical vs logical constness

Explain the difference between physical and logical constness. Where does **mutable** fit in?

C6–Q4: Include guards and ODR

Why do we need include guards? What is the One Definition Rule problem they help prevent?

C6–Q5: Forward declarations

What problem do forward declarations solve? Give a small example where they avoid cyclic includes.

C6–Q6: Class vs struct

In C++, what is the difference between **class** and **struct**? When is a struct appropriate?

C6–Q7: Localization of change

Explain what *localization of change* means and how encapsulation helps achieve it.

Solutions

C6–A1

Interface = public members visible to users. ADT means the interface promises behavior/properties (pre/post-conditions, invariants) while hiding implementation details.

C6–A2

Invariant = condition that must always hold for a valid object between public calls. Example: for a **Fraction**, denominator is never 0; for a **BankAccount**, balance never goes below an allowed limit.

C6–A3

Physical constness: no bytes of the object change. Logical constness: observable behavior does not change, even if some cache/bookkeeping changes internally. **mutable** allows changing specific members inside **const** methods to support logical constness (e.g., cached values, logging buffer).

C6–A4

Include guards prevent multiple inclusion of the same header in one translation unit. Without them you can violate the One Definition Rule (duplicate class/function definitions) or get redefinition compiler errors.

C6–A5

Forward declaration tells the compiler a type exists without including its full definition. This breaks cyclic includes and reduces compile time. Example: `class B;` in `A.h` if `A` stores `B*` or `B&`.

C6–A6

Only default access differs: `struct` members default to public, `class` defaults to private. Use `struct` for plain data aggregates where public members make sense.

C6–A7

Localization of change means you can change implementation without changing users of the class. Encapsulation (private data + stable public interface) lets you refactor internals while keeping the same interface.

7 Chapter 7: Class Derivation

Questions

C7–Q1: Subclassing vs subtyping

Explain the difference between subclassing and subtyping. Are they always the same in C++?

C7–Q2: Construction/destruction order

For a derived class object, in what order do constructors and destructors run (base vs derived)? Why?

C7–Q3: Virtual table (vtable)

What is a vtable (high-level)? What problem does it solve?

C7–Q4: Object slicing

What is slicing? Explain what happens here:

```
Base b = Derived{};
```

C7–Q5: Hiding in inheritance

Define **hiding**. Show a short example where a derived function hides a base overload.

C7–Q6: Access control in inheritance

What is the difference between `class D : public B` and `class D : private B` in terms of member access through `D`?

C7–Q7: Virtual destructor

Why should a base class used polymorphically have a virtual destructor? What can go wrong if it does not?

Solutions

C7-A1

Subclassing is a code-reuse mechanism (inherit members). Subtyping is substitutability (can use derived where base is expected). In C++, public inheritance is intended to model subtyping; private inheritance is usually reuse without subtyping.

C7-A2

Construction: base $\rightarrow$ derived, so base part is initialized first. Destruction: derived $\rightarrow$ base, so derived cleans up before base.

C7-A3

A vtable is a runtime table of function pointers used for virtual dispatch. It allows calling the correct overridden method based on the dynamic type when you call via base pointer/reference.

C7-A4

Slicing means copying only the base subobject into a base object; derived-specific state is lost. `Base b = Derived{};` constructs `b` as a pure `Base` with no derived part.

C7-A5

Hiding: a derived declaration with the same name can hide base overloads.

```
struct B { void f(int); };
struct D : B { void f(double); }; // hides B::f(int)
```

Fix: `using B::f;` in `D`.

C7-A6

With public inheritance, public members of `B` stay public in `D`. With private inheritance, public/protected members of `B` become private in `D` (not accessible through `D` by users).

C7-A7

Deleting via base pointer must call derived destructor too. Without virtual destructor, only base destructor runs $\rightarrow$ resource leak/UB if derived owns resources.

8 Chapter 8: Design Patterns

Questions

C8-Q1: SRP and OCP

Define **Single Responsibility Principle** and **Open-Closed Principle**. Give one small example design change that violates each.

C8-Q2: Liskov Substitution Principle (LSP)

State LSP in your own words. Give one example of a violation.

C8-Q3: Factory Method

What problem does the Factory Method pattern solve? Give a minimal example of using a factory to avoid `if/else` on concrete types.

C8–Q4: Abstract Factory

What extra problem does Abstract Factory solve compared to Factory Method?

C8–Q5: Observer

Describe the Observer pattern: who are Subject and Observers, and what is the main benefit?

C8–Q6: Command

What is the Command pattern useful for? Mention one practical application (undo/redo, UI buttons, etc.).

C8–Q7: MVC

Describe Model–View–Controller and how it separates responsibilities.

Solutions**C8–A1**

SRP: each class should have one reason to change. Violation: **FileManager** both reads and writes and also parses formats. OCP: open for extension, closed for modification. Violation: adding a new file format requires editing a long chain of **if/else** inside the existing reader.

C8–A2

LSP: if **D** is a subtype of **B**, you should be able to replace **B** with **D** without breaking correctness. Violation example: Derived strengthens preconditions or breaks base postconditions/invariants (e.g., a **Square** overriding **Rectangle::setWidth** in a way that breaks client expectations).

C8–A3

Factory Method centralizes object creation behind a function/class so client code depends on an interface, not concrete types. Example: `std::unique_ptr<Shape> makeShape(std::string type);` returns **Circle** or **Rectangle** without exposing **new** everywhere.

C8–A4

Abstract Factory creates *families* of related objects that must work together (e.g., GUI widgets for Windows vs Linux). It ensures compatibility by using one factory per family.

C8–A5

Subject holds state and notifies observers when it changes. Observers register/unregister and receive updates. Benefit: loose coupling: subject does not need to know concrete observer details.

C8–A6

Command encapsulates a request as an object (execute/undo), decoupling sender from receiver. Useful for undo/redo stacks, menu actions, button callbacks.

C8–A7

Model = data + rules, View = UI rendering, Controller = user input handling that updates model. Separation improves maintainability and testability.

9 Chapter 9: Multi-threading

Questions

C9–Q1: Thread basics

What is a thread? How do you start a thread with (a) a free function and (b) a lambda?

C9–Q2: join vs detach

Explain `join()` and `detach()`. When is `detach()` dangerous?

C9–Q3: Data race vs race condition

Define **data race** and **race condition**. Can you have one without the other?

C9–Q4: Mutex and RAII locking

What does a mutex protect? Why is `std::lock_guard` preferred over manually calling `lock()` / `unlock()`?

C9–Q5: Deadlock

What is deadlock? Give one typical cause and one practical prevention rule.

C9–Q6: Atomics

What does `std::atomic<T>` guarantee compared to a normal `T`? Give one example where an atomic counter is appropriate.

C9–Q7: thread_local storage duration

What is `thread_local`? How does its storage duration differ from static and automatic?

Solutions

C9–A1

A thread is an execution context (instruction stream + registers + stack, etc.). Start with `std::thread t(f, args...);` or `std::thread t([&]{ /* work */ });`.

C9–A2

`join()` waits until the thread finishes. `detach()` lets it run independently. Detach is dangerous if the thread uses references/pointers to objects that may die early (dangling access), or if you need deterministic shutdown.

C9–A3

Data race: unsynchronized concurrent access to same memory location where at least one access is a write → undefined behavior. Race condition: result depends on timing/interleaving (may happen even without UB). You can have a race condition without data race (e.g., two threads compete to do a task but all accesses are atomic).

C9–A4

Mutex ensures only one thread enters a critical section to protect shared data. `lock_guard` is RAII: unlock happens automatically even if exceptions/early returns occur, preventing forgotten `unlock()`.

C9–A5

Deadlock: threads wait forever for each other (cycle of locks). Cause: locking multiple mutexes in different order. Prevention: lock in a consistent global order or use `std::scoped_lock` to lock multiple at once.

C9–A6

Atomics provide atomic read-modify-write and prevent data races on that variable (with defined memory ordering rules). Example: shared counter of processed tasks.

C9–A7

`thread_local` gives each thread its own instance. Storage duration is from thread start to thread end (not program-wide like static, and not scope-bound like automatic).

10 Chapter 10: Generic Programming

Questions

C10–Q1: What is generic programming?

Explain generic programming in one paragraph. What is the core idea compared to writing many overloads manually?

C10–Q2: Template instantiation

Why does the compiler not generate code for a template until it is instantiated? Give two ways instantiation can happen.

C10–Q3: Template argument deduction

Explain deduction via call arguments. Why does this fail?

```
template<class T>
T sum(T a, T b){ return a + b; }

auto x = sum(5, 8.5);
```

Show one correct fix.

C10–Q4: Explicit specification vs deduction

Show (with a short code fragment) the difference between explicitly specifying template arguments and letting them be deduced.

C10–Q5: Explicit specialization

Why can explicit specialization be useful? Give one realistic example where a specific type needs a different implementation.

C10–Q6: Concept checking (high-level)

What is the advantage of concept checking (or constraints) for templates? Mention one benefit for users and one for the compiler/errors.

C10–Q7: Polymorphism: OOP vs generic

Describe the main difference between object-oriented polymorphism and generic (template) polymorphism.

Solutions**C10–A1**

Generic programming abstracts algorithms and data structures to work with many types without rewriting code. Instead of multiple overloads (int/double/long), you write one template and let the compiler generate type-specific versions.

C10–A2

Templates are compile-time blueprints; code is generated when instantiated. Instantiation happens (1) implicitly by calling the template with concrete types, and (2) explicitly via an explicit instantiation declaration/definition.

C10–A3

Deduction tries to find one `T` matching both arguments; here you have `int` and `double`, so no single `T` fits. Fix: specify `T` explicitly: `auto x = sum<double>(5, 8.5);` or redesign template to accept two types.

C10–A4

```
// explicit
auto a = sum<double>(5, 8.5);

// deduced
auto b = sum(5, 8); // T deduced as int
```

Explicit form bypasses deduction and forces the chosen type.

C10–A5

Specialization lets you handle special cases efficiently/correctly (e.g., optimize `vector<bool>` or handle floating-point rounding differently). It can also fix behavior for types that need a custom approach.

C10–A6

Concept checking/constraints ensures template arguments meet requirements (e.g., *must be addable*). Benefit for users: clearer, earlier error messages. Benefit for compiler: avoids instantiating invalid code paths and improves diagnostics.

C10–A7

OOP polymorphism is usually runtime (virtual functions + vtable) and works via base pointers/references. Generic polymorphism is compile-time (templates) where the compiler generates code per type; no runtime dispatch is needed.